

Day 08: Probability & Simulation - Python Code Blueprint

PHASE 3: CORE MANUAL LOOPS & ADVANCED NUMPY GRID IMPLEMENTATIONS

Note For Copy-Paste: All codes are completely written out with absolute clarity. You can easily copy and paste them directly into your Jupyter Notebook or VS Code editor.

कॉपी-पेस्ट के लिए निर्देश: सभी कोड्स को पूरी तरह से बिना किसी शॉर्टकट के साफ-साफ लिखा गया है। आप इन्हें आसानी से कॉपी करके सीधे ज़ुपिटर नोटबुक या वीएस कोड में चला सकते हैं।

1. Addition Rule ("OR" Logic) - Practical Implementations

A) Pure Manual Loop Code (Zero Library Method)

A) प्योर मैनुअल लूप कोड (बिना किसी बाहरी लाइब्रेरी के काम करने का तरीका)

```
# =====
# DAY 08: ADDITION RULE ("OR") PURE MANUAL CODE
# =====

# 1. Dice ka total Sample Space (Ludo ke paase par jo jo aa sakta hai)
dice_sample_space = [1, 2, 3, 4, 5, 6]

# Master Formula ka neeche waala hissa: Total kitne outcomes hain (6)
total_possible_outcomes = len(dice_sample_space)

# 2. Variable banate hain favorable outcomes (jo lana hai) ko ginn ne ke liye
favorable_outcomes_count = 0

# 3. Pure Manual Loop: Sample space ke ek-ek number ko baari-baari check karenge
for outcome in dice_sample_space:

    # Condition: Hamein ya toh number 3 chahiye OR number 5 chahiye
    # 'or' ka matlab hai dono mein se koi ek bhi sach hua toh loop andar chala jayega
    if outcome == 3 or outcome == 5:
        favorable_outcomes_count += 1 # Match hote hi counter mein 1 jod do

# 4. Apply Master Formula: Jo lana hai (Upar) / Jo kul aa sakta hai (Neeche)
manual_probability = favorable_outcomes_count / total_possible_outcomes

# =====
# RESULTS DISPLAY
# =====
print("===== MANUAL ADDITION RESULTS =====")
print(f"Total Possible Outcomes (Neeche) : {total_possible_outcomes}")
print(f"Favorable Outcomes Found (Upar)   : {favorable_outcomes_count}")
print(f"Calculated Probability                : {manual_probability:.4f}")
print(f"Percentage Chance                    : {manual_probability * 100:.2f}%")
print("=====")
```

B) Advanced Vectorized Simulation Code (SciPy / NumPy Method)

B) एडवांस्ड वेक्टराइज्ड सिमुलेशन कोड (NumPy लाइब्रेरी का उपयोग करके तेज तरीका)

```
import numpy as np

# Total kitni baar paasa phenkna hai
total_trials = 100000

# Ek line mein 1 lakh random rolls taiyar! (1 se 7 boundary rule ke sath)
rolls = np.random.randint(1, 7, size=total_trials)

# Check kiya ki kahan-kahan 3 aaya OR 5 aaya, aur unko jod (sum) liya
target_hits = np.sum((rolls == 3) | (rolls == 5))

# Final Probability calculation
scipy_probability = target_hits / total_trials

print("===== SCIPY/NUMPY ADDITION SHORT RESULT =====")
print(f"Simulation Percentage : {scipy_probability * 100:.2f}%")
print("=====")
```

Deep Concept Explanation - Upper Boundary Rule (1, 7)

गहरी समझ - अपर बाउंड्री रूल: NumPy में `randint(1, 7)` लिखते हैं क्योंकि आखिरी नंबर (7) एक्सक्लूसिव होता है, यानी कंप्यूटर 7 से एक पहले रुक कर सिर्फ 1 से 6 तक के नंबर ही बनाएगा।

2. Multiplication Rule ("AND" Logic) - Practical Implementations

A) Pure Manual Nested Loop Code (Zero Library Method)

A) प्योर मैनुअल नेस्टेड लूप कोड (बिना किसी लाइब्रेरी के कॉम्बिनेशन्स बनाने का तरीका)

```
# =====
# DAY 08: MULTIPLICATION RULE PURE MANUAL CODE (Zero Library Way)
# =====

# 1. Dono Independent Events ke alag-alag Sample Spaces
coin_space = ['Head', 'Tail']
dice_space = [1, 2, 3, 4, 5, 6]

# 2. Variables set karte hain counting ke liye
total_combinations = 0
favorable_combinations = 0

# 3. Nested Loops: Ek-ek karke saare combinations banayenge
for coin_outcome in coin_space:      # Yeh loop 2 baar chalega
    for dice_outcome in dice_space:  # Yeh loop 6 baar chalega

        # Har baar ek naya combination banega, toh total count badhao
        total_combinations += 1

        # Condition Check: Hamein Sikka par 'Head' AND Paasa par exact 6 chahiye
        if coin_outcome == 'Head' and dice_outcome == 6:
            favorable_combinations += 1

# 4. Master Formula Apply
```

```

manual_multiplication_prob = favorable_combinations / total_combinations

# =====
# RESULTS DISPLAY
# =====
print("===== MULTIPLICATION MANUAL RESULTS =====")
print(f"Total Combined Outcomes (Neeche) : {total_combinations}")
print(f"Favorable Combination (Upar)      : {favorable_combinations}")
print(f"Calculated Probability              : {manual_multiplication_prob:.4f}")
print(f"Percentage Chance                   : {manual_multiplication_prob * 100:.2f}%")
print("=====")

```

B) Advanced Matrix Grid Code (NumPy meshgrid Method)

B) एडवांस्ड मैट्रिक्स ग्रिड कोड (NumPy meshgrid का उपयोग करके बिना लूप का शॉर्टकट)

```

import numpy as np

# =====
# DAY 08: MULTIPLICATION RULE SHORT CODE (NumPy Matrix Way)
# =====

# 1. Dono spaces ko numbers mein set karte hain (Sikka: 1=Head, 0=Tail | Paasa: 1 se 6)
coin_options = np.array([1, 0])
dice_options = np.arange(1, 7) # Yeh automatic array bana dega [1, 2, 3, 4, 5, 6]

# 2. NumPy Ka Jaadu: Bina loop ke saare combinations ek sath generate!
coin_matrix, dice_matrix = np.meshgrid(coin_options, dice_options)

# 3. Condition Check: Hamein Sikka par Head (1) AND Paasa par 6 chahiye
# NumPy mein 'AND' ke liye '&' ka use hota hai
target_hits = np.sum((coin_matrix == 1) & (dice_matrix == 6))

# 4. Total outcomes nikalte hain matrix ka size dekh kar (Dono me se koi bhi le sakte hain)
total_combinations = coin_matrix.size

# 5. Master Formula Apply
short_multiplication_prob = target_hits / total_combinations

# =====
# RESULTS DISPLAY
# =====
print("===== MULTIPLICATION SHORT RESULTS =====")
print(f"Total Combined Outcomes (Neeche) : {total_combinations}")
print(f"Favorable Combination (Upar)      : {target_hits}")
print(f"Calculated Percentage              : {short_multiplication_prob * 100:.2f}%")
print("=====")

```

3. Complete Matrix Post-Mortem Visual Structures

Behind the scenes, `np.meshgrid()` creates these full arrays for simultaneous element-by-element comparison:

बैकग्राउंड में `np.meshgrid()` इन दोनों पूरी मैट्रिसेस को आमने-सामने तुलना के लिए इस प्रकार तैयार करता है:

A) Full Coin Matrix Structure (coin_matrix)

```
coin_matrix = [  
    [1, 0], # Row 1 (Dice ke number 1 ke samne - Head aur Tail)  
    [1, 0], # Row 2 (Dice ke number 2 ke samne - Head aur Tail)  
    [1, 0], # Row 3 (Dice ke number 3 ke samne - Head aur Tail)  
    [1, 0], # Row 4 (Dice ke number 4 ke samne - Head aur Tail)  
    [1, 0], # Row 5 (Dice ke number 5 ke samne - Head aur Tail)  
    [1, 0] # Row 6 (Dice ke number 6 ke samne - Head aur Tail)  
]
```

B) Full Dice Matrix Structure (dice_matrix)

```
dice_matrix = [  
    [1, 1], # Row 1 (Dono columns mein number 1 stretched)  
    [2, 2], # Row 2 (Dono columns mein number 2 stretched)  
    [3, 3], # Row 3 (Dono columns mein number 3 stretched)  
    [4, 4], # Row 4 (Dono columns mein number 4 stretched)  
    [5, 5], # Row 5 (Dono columns mein number 5 stretched)  
    [6, 6] # Row 6 (Dono columns mein number 6 stretched)  
]
```

Why coin_matrix.size or dice_matrix.size works equally:

चाहें coin_matrix.size लें या dice_matrix.size, दोनों ही कुल 12 कॉम्बिनेशन्स (6 Rows * 2 Columns) का सटीक काउंट वापस देंगे, क्योंकि दोनों ग्रीड्स एक दूसरे के ऊपर बिल्कुल बराबर ओवरलैप करती हैं।